

Introspective Starts

Starting a section agent should begin with an explicit inspection pass rather than immediate prose generation. In practice, that means the agent first reads the full document outline to recover the chapter-level argument, then checks the selected section against its sibling sections so the local transition fits the surrounding narrative. It also inspects the section payload itself, the registered reference set, and any diagram opportunities before it writes a draft. This front-loaded review keeps the section aligned with the book’s schema-first workflow and prevents the agent from drafting in isolation.

The inspection pass is not merely passive reading. It is a deliberate inventory of the work state that the agent will need in order to act responsibly: the current outline node, the section summary, any imported source material, the available citation keys, and the visual affordances of the section. For a section agent, those inputs define the boundaries of what can be stated directly, what must be deferred to a reference handoff, and whether a diagram would materially improve comprehension. The result is a small but important planning step that turns a generic start-up event into a concrete work plan.

That planning step is especially important because the section agent is not starting from a blank page in the abstract; it is starting from a specific place in a larger manuscript. The chapter has already established the agent roles, the task queue model, and the message-and-callback runtime. An introspective start extends that machinery into the first moment of section work by making the agent ask, in effect, “What is the document trying to do, what is this section supposed to contribute, and what support is already available?” Only after those questions are answered does drafting become a responsible next step.

After that inspection, the agent queues a self-addressed plan message. This message enters the same durable task system used for ordinary work, so the plan is visible, inspectable, and replayable like any other task. The self-message can summarize the intended drafting strategy, note missing references, record whether a diagram request is warranted, and identify the next action to execute. In other words, the agent does not silently “think ahead”; it externalizes the plan into the queue so the runtime can track how the section moved from initialization to drafting.

That pattern matters because it makes section startup part of the manuscript’s observable state. A section does not begin as an opaque burst of generation; it begins as a documented decision to inspect, assess, and then act. The introspective start therefore serves two purposes at once: it gives the section agent enough context to draft well, and it leaves behind a durable trace that explains why the draft took the shape it did. In a codynamic system, even the first step of writing is itself a managed artifact.

The section also fits naturally beside the surrounding runtime discussion. The preceding sections explain how tasks are queued and how messages move through the system; this section narrows that machinery to the moment a section agent wakes up and decides what to do first. The emphasis is not on a special new capability, but on disciplined sequencing: inspect the document, inspect the section, inspect the available support, then queue the plan that will govern the next action.

Seen that way, the introspective start is a small control loop rather than a ceremonial preface. It reduces the chance that a section agent will draft against stale assumptions, overlook a sibling section’s contribution, or miss a missing citation that should have been routed elsewhere. It also makes the agent’s first move legible to the rest of the system: the hypervisor can see that the section has entered a planning phase, the gardener can later interpret the resulting draft in light of the inspection, and the references and diagram workflows can be invoked only when the section actually needs them.

The section is intentionally self-contained and does not require a visual to carry its argument,

so no diagram is requested here.